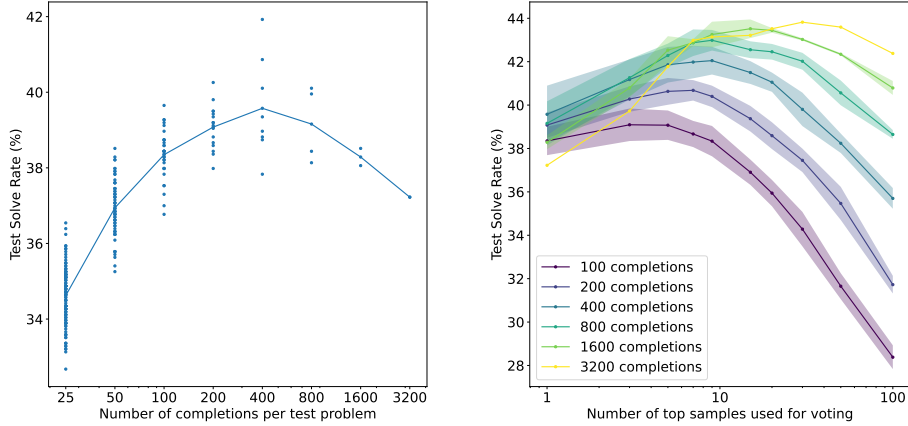




(a) 6B verification test performance when given varying numbers of completions per problem to rank.

(b) 6B verification test performance when varying the number of top ranked samples allowed to vote on the answer.

Figure 7: Performance as the amount of test time compute varies.

sense: better understanding this language distribution should only aid the verifier in discriminating between samples.

In Figure 6c, we separately ablate the model size of the generator and the verifier. We find that using a large generator with a small verifier performs significantly better than using a small generator with a large verifier. Verification is still remarkably effective, even when the verifier is much smaller than the generator. This suggests that the verifier may often be relying on relatively coarse heuristics to discriminate between solutions from a given generator, rather than attempting a more thorough form of verification.

5 Additional Experiments

5.1 Test Time Compute

At test time, we can choose to generate arbitrarily many solutions to be judged by the verifier before selecting the highest ranked completion. Figure 7a shows how 6B verifier performance varies with the number of completions per test problem. At this scale, performance improves as we increase the number of completions up to 400. Beyond this point, performance start to decrease. This suggests that the benefits of search are eventually outweighed by the risk of finding adversarial solutions that fool the verifier. In general, we evaluate verifier test performance using 100 completions, since this captures most of the benefits of verification with a relatively modest compute cost.

To further increase performance, we can take a majority vote among the top verifier-ranked solutions instead of selecting only the single top solution.

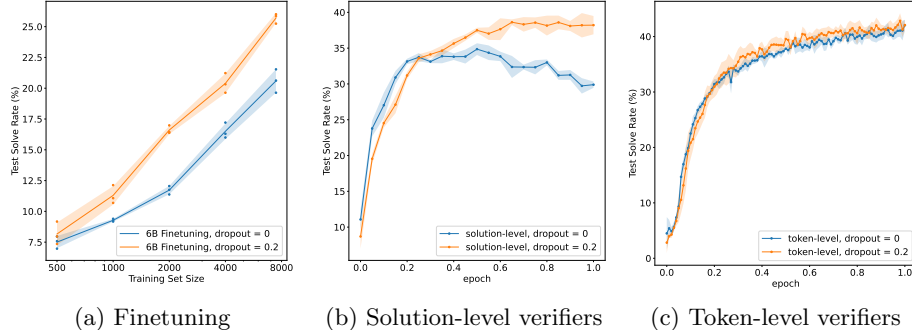


Figure 8: 6B finetuning and verification dropout ablations.

This voting process considers only the final answer reached by the individual solutions: the final answer selected is the one with the most votes. Figure 7b shows how performance varies as we allow a greater number of top samples to cast a vote. Unsurprisingly, when starting with a greater number of samples, we can afford to allow a greater number of samples to cast a vote. When we have only 100 samples, it is optimal to allow only the top 3-5 samples to cast a vote. When we have 3200 samples, it is approximately optimal to allow the top 30 to cast a vote.

5.2 Regularization

We find that both finetuning and verification strongly benefit from the use of dropout as a regularizer. Specifically, we apply residual dropout (Vaswani et al., 2017) along the residual paths of each layer in the network. We use 20% dropout for all dropout experiments, chosen based on the results of a hyperparameters sweep. We note that GPT-3 models are not pretrained with dropout. For experiments involving dropout, we therefore perform additional pretraining with dropout before subsequently finetuning the models. This mitigates the distribution shift the model experiences during finetuning.

We first investigate the effect of dropout on finetuning across various training set sizes. Figure 8a shows that dropout leads to a significant improvement over baseline. We next investigate the effect of dropout on verifiers, considering both the solution-level and token-level variants. In Figure 8b, we see that dropout significantly improves solution-level verifiers, mitigating the overfitting that occurs in the unregularized baseline. Notably, using dropout with solution-level verifiers reaches a similar level of performance as token-level verifiers. In Figure 8c, we apply dropout to token-level verifiers. Since token-level verifiers are already less susceptible to overfitting, it is no surprise that the impact of dropout is less significant. Nevertheless, we do still see a slight gain from training token-level verifiers with dropout. Note that we increase the batch size for token-level verifiers by a factor of 4, to better handle the more difficult objective and the noise from dropout.

6 Conclusion

We have seen that verification provides a significant performance boost relative to a finetuning baseline. On the full dataset, 6B verification slightly outperforms a finetuned 175B model, thereby offering a boost approximately equivalent to a 30x model size increase. We have also seen that token-level verifiers are less prone to overfitting than solution-level verifiers, and that all methods benefit from regularization with residual dropout. We expect verification to scale well to problem distributions that require more complex mathematical reasoning, and we hope GSM8K supports the development of new methods that scale even better.

Acknowledgements

We thank Dan Hendrycks, Leo Gao, Alec Radford, and Giambattista Parascandolo for their valuable feedback on this paper; Harri Edwards, Yura Burda, Michael Wu, and Nick Ryder for many insightful conversations; Michael Petrov, Alethea Power, and Jacob Jackson for their technical assistance; the OpenAI Supercomputing team for the infrastructure that made these experiments possible; and the team at Surge AI for performing the GSM8K data collection.

References

- A. Amini, S. Gabriel, P. Lin, R. Koncel-Kedziorski, Y. Choi, and H. Hajishirzi. Mathqa: Towards interpretable math word problem solving with operation-based formalisms. *arXiv preprint arXiv:1905.13319*, 2019.
- T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Nee-lakantan, P. Shyam, G. Sastry, A. Askell, et al. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165*, 2020.
- K. Chen, Q. Huang, H. Palangi, P. Smolensky, K. D. Forbus, and J. Gao. Mapping natural-language problems to formal-language solutions using structured neural representations. In *ICML*, 2020.
- X. Chen, C. Liang, A. W. Yu, D. Zhou, D. Song, and Q. V. Le. Neural symbolic reader: Scalable integration of distributed and symbolic representations for reading comprehension. In *International Conference on Learning Representations*, 2019.
- T.-R. Chiang and Y.-N. Chen. Semantically-aligned equation generation for solving and reasoning math word problems. *arXiv preprint arXiv:1811.00720*, 2018.
- D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.